CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 15: Analyse Observability Challenges

Exam objective: Analyse observability challenges in Claude-based agentic systems, including distributed tracing, structured logging, metrics and dashboards, correlation IDs, tool execution traces, agent execution monitoring, token consumption tracking, latency percentiles, error rates and failure monitoring, and privacy-aware logging.

Q1 **A telecom provider's Claude agent orchestrates calls across a retrieval service, a billing API, and a notification service to resolve a single customer request. When a customer complains about an incorrect response, engineers cannot reconstruct which service contributed what. What observability capability is missing?**

A Prompt caching

B **Distributed tracing with a shared trace/correlation ID propagated across all services involved in handling the request**

C A larger context window

D Reranking of retrieved documents

CORRECT ANSWER **B**

WHY Distributed tracing with a propagated correlation ID is exactly what lets engineers reconstruct the full path of a request across multiple services and tools, attributing behavior to the right component. Prompt caching, context window size, and reranking are unrelated to the ability to trace a multi-service request end-to-end.

Q2 **A healthcare documentation agent logs full patient conversation transcripts, including names and diagnoses, in plaintext to a centralized logging system accessible to the entire engineering organization. What observability principle is being violated?**

A Latency percentile tracking

B **Privacy-aware logging, which requires redacting, masking, or restricting access to sensitive data captured in logs**

C Tool execution tracing

D Metrics dashboard granularity

CORRECT ANSWER B

WHY Privacy-aware logging requires minimizing, redacting, or access-restricting sensitive data (like PHI) captured for observability purposes, since broad plaintext logging of identifiable health data to a wide audience creates serious compliance and privacy risk. This scenario is not about latency measurement, tool tracing, or dashboard granularity — it is specifically about unsafe handling of sensitive content in logs.

Q3 An e-commerce company's dashboards show average response latency of 800ms, which looks healthy, but customer complaints about slow responses are rising. What is the most likely observability gap?

A. **The dashboard lacks latency percentile breakdowns (e.g., p95, p99), so tail latency problems affecting a subset of users are hidden by the average**

B. The dashboard is using the wrong color scheme

C. The dashboard needs to track token consumption instead of latency

D. The issue is unrelated to observability and purely a customer perception problem

CORRECT ANSWER A

WHY Averages can mask tail latency problems; without percentile breakdowns, a system can look healthy on paper while a meaningful fraction of real requests experience significant delays, which matches the complaint pattern described. Color scheme and token-consumption tracking do not address the specific gap of hidden tail latency, and dismissing it as "just perception" ignores a measurable, fixable observability shortfall.

Q4 **A manufacturing company's agent occasionally fails a tool call (e.g., sensor API timeout) but silently returns a generic "no data available" message to the user without logging the underlying failure reason or raising an alert. What should be added?**

A Nothing; user-facing messages should never reveal technical detail

B **Structured error logging capturing the failure type/cause at the point of the tool call, combined with error-rate monitoring and alerting so recurring failures are surfaced to engineers, independent of what the user sees**

C A longer system prompt instructing the model to apologize more clearly

D Switching to a model with higher reasoning effort to avoid tool failures

CORRECT ANSWER B

WHY Observability requires capturing the actual failure cause in structured logs and tracking error rates/alerts even when the user-facing message is intentionally simplified; engineers need visibility into failure patterns to detect and fix root causes. Hiding user-facing detail is reasonable, but it must not come at the cost of losing internal diagnostic visibility, and neither prompt wording nor reasoning effort addresses an infrastructure-level tool failure.

Q5 An architect designing monitoring for a multi-agent customer support system wants to track, for every end-user request, how many tokens were consumed across the orchestrator and each sub-agent, broken down by model tier used. What is the primary business value of this specific metric?

A It is required for GDPR compliance in all cases

B It enables cost attribution and capacity planning, helping the organization understand and control spend across different agents/model tiers and identify unexpectedly expensive request patterns

C It eliminates the need for any latency monitoring

D It automatically improves the accuracy of model outputs

CORRECT ANSWER B

WHY Token consumption tracking broken down by agent/model tier is primarily a cost-attribution and capacity-planning tool, letting organizations see where spend concentrates and catch inefficient or runaway usage patterns. It is not a GDPR-specific requirement, it does not replace latency monitoring (a separate concern), and tracking token usage has no direct effect on output accuracy.

Q6 **A government benefits agency's Claude-based assistant handles thousands of concurrent sessions. During an incident, engineers want to isolate the full sequence of tool calls, model requests, and retrieval operations for one specific affected citizen's session without wading through unrelated logs. What enables this?**

A Reducing the number of tools available to the agent

B **A consistent correlation/session ID attached to every log entry, trace span, and tool execution record generated during that session, enabling precise filtering**

C Increasing the context window so more history fits in a single prompt

D Switching the agent to a stateless configuration

CORRECT ANSWER B

WHY Correlation IDs consistently propagated through logs, traces, and tool execution records are exactly what allow engineers to filter and reconstruct a single session's full activity amid massive concurrent volume. Reducing tool count, expanding context windows, and statelessness are unrelated to the ability to isolate and filter a specific session's observability data.

Q7 **A SaaS company's on-call engineer is paged at 2 a.m. because the agent's error rate spiked, but the alert gives no indication of which specific tool or dependency is failing, only an aggregate "errors increased" signal. What observability improvement would most reduce mean-time-to-resolution?**

A Remove the alert entirely since it is not actionable

B **Break down error-rate metrics and alerting by dimension (e.g., per tool, per downstream service, per error type) so the on-call engineer can immediately identify the likely failing component**

C Increase the alert threshold so it pages less often

D Replace the alert with a daily summary email instead of real-time paging

CORRECT ANSWER **B**

WHY Dimensional breakdown of error-rate metrics (by tool, dependency, or error type) turns a vague "something is wrong" signal into an actionable one, directly reducing diagnosis time during an incident. Removing the alert or delaying it to a daily email would harm incident response, and simply raising the threshold risks missing real problems rather than making the signal more useful.

Q8 **A logistics company wants to monitor whether its route-planning agent's decisions are deteriorating over time (e.g., gradually worse route choices) rather than failing outright with errors. Which observability approach is most appropriate?**

A. Rely only on binary success/failure logging, since any non-error outcome is by definition acceptable

B. Track quality-oriented metrics (e.g., route efficiency score, user override rate, customer satisfaction signals) over time on a dashboard, in addition to standard error-rate and latency metrics, to catch silent quality degradation

C. Disable monitoring for successful requests to reduce log volume

D. Only monitor token consumption, since cost is the primary concern

CORRECT ANSWER B

WHY Silent quality degradation (the agent "succeeding" technically but producing worse decisions) requires outcome/quality metrics beyond simple error rates — this is a known observability gap in agentic systems where failures aren't always loud exceptions. Binary success/failure logging alone would miss this entirely, disabling monitoring on successes removes the very data needed to detect drift, and token consumption tracks cost, not decision quality.

Q9 **An architect is building a tool execution trace schema for an agent that calls external APIs. Besides timestamps and success/failure status, which additional field is most valuable for diagnosing why a specific tool call produced an unexpected result, while still respecting data minimization?**

A. The full raw request and response payload for every call, always stored indefinitely with no redaction

B. **A structured record of the tool name, sanitized/redacted input parameters, response status, and duration — retaining enough detail to diagnose issues without capturing unnecessary sensitive payload content**

C. Only the tool name, with no other call details

D. The user's full session transcript embedded directly into every individual tool trace record

CORRECT ANSWER B

WHY Effective tool execution traces balance diagnostic usefulness with data minimization — capturing tool name, sanitized parameters, status, and duration gives engineers enough to debug issues without indefinitely retaining full, potentially sensitive payloads. Storing everything indefinitely without redaction creates privacy/compliance risk, logging only the tool name is insufficient for diagnosis, and embedding full session transcripts into every trace record is excessive and privacy-unfriendly.

Q10 **A financial institution's compliance team requires that all agent tool calls touching account data be auditable for at least seven years, but the engineering team's current logs are stored in a system with a 30-day retention policy optimized for debugging. What is the correct resolution?**

A Extend the 30-day debugging log retention to seven years for all logs, including verbose debug-level traces

B **Separate audit-relevant events (e.g., account data access, tool calls with financial impact) into a dedicated, immutable audit log store meeting the compliance retention requirement, distinct from short-lived operational debug logs**

C Tell the compliance team that observability logs cannot serve compliance purposes under any circumstances

D Store the seven-year audit requirement only in a spreadsheet maintained manually by an analyst

CORRECT ANSWER **B**

WHY Compliance-grade audit trails typically require a distinct, immutable, long-retention store for specific auditable events, separate from high-volume, short-retention operational/debug logs which are optimized for a different purpose (troubleshooting, not audit). Extending full debug-log retention to seven years is costly and includes irrelevant noise, dismissing the requirement is not a valid architectural response, and manual spreadsheet tracking does not scale or meet audit integrity standards.

Q11 An architect reviewing dashboards for a multi-agent insurance claims system notices that metrics are aggregated only at the overall system level, with no way to see per-agent (e.g., fraud-detection sub-agent vs. document-extraction sub-agent) latency or error contribution. What is the risk of this design?

A. There is no risk; system-level aggregates are always sufficient

B. A single misbehaving sub-agent's problems can be masked by the aggregate metrics of better-performing sub-agents, delaying detection and diagnosis of the actual source of degradation

C. Aggregated metrics always provide better accuracy than per-agent metrics

D. This design improves privacy by hiding which sub-agent is responsible

CORRECT ANSWER B

WHY Aggregate-only metrics can hide a localized problem in one sub-agent behind the averaged performance of others, which is a real and common observability pitfall in multi-agent systems — granular, per-component metrics are needed to pinpoint root cause quickly. Aggregation does not improve underlying metric accuracy, and hiding which component is responsible is a drawback for diagnosis, not a meaningful privacy benefit (agent identity is an internal system detail, not sensitive user data).

Q12 **A retail chatbot team wants to detect, in near real time, when the agent's tool-calling success rate for its "process_return" tool drops below 90%, which historically indicates an upstream API problem. What combination of observability components should be implemented?**

A. A manual weekly report reviewed by an engineer

B. A metric tracking success/failure counts specifically for the `process_return` tool, visualized on a dashboard, with an automated threshold-based alert that pages the team when the rate drops below 90%

C. A single log line printed to console output with no aggregation or alerting

D. Relying on customer complaints as the sole detection mechanism

CORRECT ANSWER B

WHY Near-real-time detection requires per-tool success-rate metrics with dashboard visibility and automated threshold alerting — this is the standard pattern for proactive failure monitoring rather than reactive discovery. Weekly manual reports and waiting for customer complaints introduce unacceptable detection delay, and unaggregated console logging provides no actionable signal at scale.

Q13 **A legal document-review agent's observability stack logs the full text of every document the agent processes, including attorney-client privileged content, to a third-party APM (application performance monitoring) vendor's cloud logging service with no data processing agreement in place. What is the most pressing issue?**

A. APM vendors are never permitted to process any customer data under any agreement

B. Sending privileged/sensitive content to a third-party service without appropriate contractual and technical safeguards (e.g., data processing agreements, redaction, data residency controls) creates significant legal and confidentiality risk

C. The issue is purely a latency concern since third-party logging can be slow

D. This is acceptable as long as the APM vendor has a public privacy policy on their website

CORRECT ANSWER B

WHY Logging privileged or sensitive content to a third party without proper agreements, redaction, or residency controls is a serious confidentiality and legal risk, and privacy-aware logging practice requires minimizing what sensitive data leaves the controlled environment, or ensuring equivalent protections apply when it does. Third-party processing can be done safely with proper agreements, so it is not categorically prohibited; the core issue is not latency; and a generic public privacy policy does not substitute for a specific data processing agreement covering this use case.

Q14 An architect wants to correlate a spike in Claude API error rates with a recent deployment of a new tool integration. Logs exist, but deployment events are tracked in a completely separate system with no shared timeline or identifiers. What should be added to close this gap?

A Nothing; correlating deployments with error spikes is not a legitimate observability use case

B Deployment/change events should be ingested into the same observability platform (or correlated via timestamps/version tags) so engineers can visually and programmatically correlate error-rate changes with specific releases

C Simply increase log verbosity across the board

D Remove the new tool integration without further investigation

CORRECT ANSWER B

WHY Correlating deployment/change events with metrics (a common SRE practice, e.g., annotating dashboards with release markers) is essential for root-causing regressions tied to specific releases, and requires the deployment timeline to be visible alongside operational metrics. Dismissing this correlation as illegitimate ignores a standard and valuable observability technique, blanket verbosity increases add noise without targeted correlation, and reverting without investigation risks losing the root cause and repeating the mistake.

Q15 **A manufacturing plant's agent monitoring dashboard shows p50 latency of 400ms and error rate of 0.5%, both seemingly healthy, yet plant operators report the agent "hangs" unpredictably during specific shift-change periods. What should the observability strategy add to catch this pattern?**

A Nothing further is needed since the existing metrics are within healthy thresholds

B **Time-windowed/segmented analysis of latency and error metrics (e.g., by hour of day, shift period) in addition to overall aggregates, to detect patterns that are only visible when sliced by time or operational context**

C Replace p50 with p99 only, discarding p50 entirely

D Ask operators to stop using the agent during shift changes instead of investigating

CORRECT ANSWER **B**

WHY Overall aggregate metrics can hide time-bound or contextual patterns (like shift-change load spikes); segmenting metrics by relevant dimensions such as time-of-day is necessary to surface problems that only manifest under specific operational conditions. Concluding "healthy" from aggregates alone misses the reported issue, swapping one percentile for another doesn't address the need for time-based segmentation, and telling operators to avoid the system sidesteps the actual engineering investigation needed.

Q16 **A SaaS company's streaming chat agent occasionally refuses to answer certain prompts, but the on-call dashboard — wired to fire alerts only on API error events — shows zero alerts during a period when refusal volume spiked noticeably. Why are the refusals invisible to this alerting setup?**

A Refusals are a client-side rendering choice and are never actually sent by the model

B **Refusals arrive on an otherwise successful response stream and fire a dedicated refusal event rather than the API error event, so alerting wired only to error events will never see them**

C The dashboard is misconfigured to ignore all events with a 200 status code

D Refusals only occur in non-streaming requests, so a streaming client would never encounter them

CORRECT ANSWER **B**

WHY A refusal is delivered as part of a normal, successful response stream and triggers a distinct refusal event rather than the API error event, so any alerting pipeline that only listens for error events will systematically miss a spike in refusals even though the underlying behavior is a genuine quality signal worth tracking. Refusals are a real model-emitted signal, not a rendering artifact; they are not confined to non-streaming requests; and the root cause here is event-channel selection, not a status-code filtering bug.

Q17 **An architect enables what they believe is full request tracing for a Claude Agent SDK deployment by setting a single environment variable, then is surprised when no spans appear in their collector. What is most likely missing, and what should a correctly instrumented trace look like once fixed?**

A Nothing is missing; traces simply take 24 hours to first appear after enabling any setting

B **Agent SDK telemetry requires both an enabling environment variable and at least one configured exporter before anything is emitted; once correctly configured, a well-formed trace shows a root interaction span per user prompt, with API calls, tool calls, and hook executions as child spans, and any subagent's spans nested under the parent tool span that delegated to it**

C The SDK always emits telemetry automatically; the collector itself must be broken

D Tracing is only available for non-streaming requests, so switching off streaming will make spans appear

CORRECT ANSWER **B**

WHY Agent SDK telemetry is off until both an enabling environment variable is set and at least one exporter is configured — a single setting is not sufficient — and the SDK itself emits no telemetry directly, instead passing configuration through to the instrumented process which records the actual spans; a correctly shaped trace has a root interaction span per prompt with API/tool/hook calls as children and subagent spans nested under the delegating tool span. Assuming a fixed delay, assuming automatic emission, or assuming a streaming-related limitation all misdiagnose what is actually a two-part enablement requirement.

Q18 **After a prompt-caching cache-miss incident, an engineer uses a cache diagnostics feature that compares the current request against a previous response ID and reports the first point of divergence among the model, system prompt, tools, and message history. The engineer fixes the one reported divergence, confirms the diagnostic no longer flags it, and closes the incident. What risk does this approach carry?**

A None; once the first divergence is fixed, the diagnostic guarantees the cache will hit on every subsequent request

B **The diagnostic reports only the earliest divergence it finds, so additional, later divergences may still exist but remain hidden behind the first one; the engineer should fix and re-run the comparison rather than fix and assume the issue is fully resolved**

C Cache diagnostics only evaluate the system prompt, so checking tools and message history separately is redundant

D Cache misses cannot be diagnosed this way at all; only full cache read token counts are informative

CORRECT ANSWER **B**

WHY Because the diagnostic feature reports only the earliest point of divergence across the model, system prompt, tools, and message history, fixing that one issue and re-checking is necessary — a clean result after a single fix does not guarantee there was only one cause, since later divergences can be masked behind the first. Treating one clean diagnostic run as final proof of resolution risks leaving additional unresolved mismatches in place, the tool explicitly considers more than just the system prompt, and it is more informative than a bare cache-read-token count dropping to zero. --